INFORMATIONSBLATT

IB02 – Bedingungen verknüpfen und Eingaben prüfen

Dynamische Webseiten mit PHP · LS04 | 2BKI2 · Berufskolleg Technische Assistenz · Schuljahr 2026/2027

🎯 Darum geht es

Sie verknüpfen Bedingungen mit UND, ODER und NICHT, prüfen mit `isset()` und `??`, ob ein Wert überhaupt vorhanden ist, und bauen eine Formularprüfung, die alle Fehler auf einmal meldet.

1 | Logische Operatoren

Operator	Name	wahr, wenn …	Beispiel
`&&`	UND	beide Bedingungen wahr sind	geöffnet: `$stunde >= 12 && $stunde < 21`
`||`	ODER	mindestens eine wahr ist	Wochenende: `$tag == 6 || $tag == 7`
`!`	NICHT	die Bedingung falsch ist	Checkbox fehlt: `!isset($_POST['dsgvo'])`

UND und ODER — beispiel.php

```php
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>Logik</title></head>
<body>

<?php
  $personen = 9;
  $tel = "";

  if ($personen >= 8 && $tel === "") {
    echo "<p>Bitte geben Sie für große Gruppen eine Telefonnummer an.</p>";
  }

  $tag = 6;   // Samstag
  if ($tag == 6 || $tag == 7) {
    echo "<p>Am Wochenende öffnen wir schon um 11 Uhr.</p>";
  }
?>

</body>
</html>
```

SO SIEHT ES IM BROWSER AUS

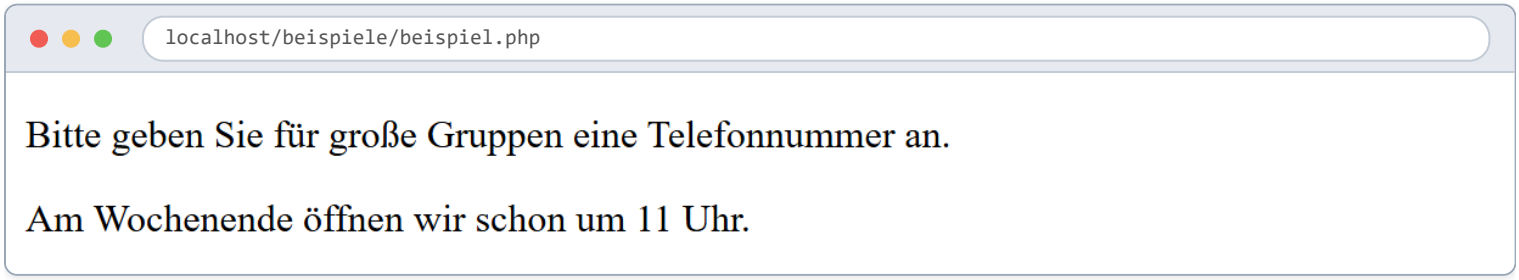


Abbildung 1: Beide Bedingungen sind erfüllt, beide Sätze erscheinen.

A	B	A && B	A || B
true	true	true	true
true	false	false	true
false	true	false	true
false	false	false	false

> ⚠️ **„ZWISCHEN“ BRAUCHT ZWEI VERGLEICHE**
>
> Mathematisch schreibt man 12 ≤ Stunde < 21. In PHP geht das nicht in einem Stück: `12 <= $stunde < 21` ist ein Fehler. Richtig: `$stunde >= 12 && $stunde < 21`.

2 | Gibt es den Wert überhaupt? isset() und ??

Fehlt ein Formularfeld – eine nicht angehakte Checkbox oder eine direkt aufgerufene Seite –, erzeugt `$_POST['…']` eine Warnung (LS03, IB01). `isset()` prüft vorher, ob der Eintrag existiert. Der Operator `??` liefert einen Ersatzwert, falls nicht:

isset() und ?? — beispiel.php

```php
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>isset</title></head>
<body>

<?php
  if (isset($_POST['dsgvo'])) {
    echo "<p>Datenschutzerklärung bestätigt.</p>";
  } else {
    echo "<p>Bitte bestätigen Sie die Datenschutzerklärung.</p>";
  }

  $anlass = $_POST['anlass'] ?? "kein besonderer";
  echo "<p>Anlass: $anlass</p>";
?>

</body>
</html>
```

SO SIEHT ES IM BROWSER AUS



Abbildung 2: Checkbox nicht angehakt, Anlass nicht gesendet – trotzdem keine Warnung.

Schreibweise	Bedeutung
`isset($_POST['dsgvo'])`	`true`, wenn der Eintrag existiert
`$_POST['anlass'] ?? "kein besonderer"`	der Wert – oder der Ersatzwert, falls es ihn nicht gibt
`trim($_POST['name'] ?? '') === ''`	Name fehlt oder besteht nur aus Leerzeichen

3 | Warum der Server prüfen muss

`required`, `min` und `max` im Formular sind bequem für ehrliche Besucher. Sie laufen aber im Browser – und den hat der Besucher unter Kontrolle. Mit den Entwicklerwerkzeugen (F12) sind die Attribute in Sekunden entfernt; Programme schicken Daten sogar ganz ohne Formular.

> 📌 **GRUNDREGEL**
>
> Prüfungen im Browser dienen dem **Komfort**. Prüfungen auf dem Server dienen der **Sicherheit**. Jede wichtige Regel gehört deshalb in den PHP-Code – auch wenn der Browser sie schon prüft.

4 | Alle Fehler auf einmal melden

Besucher ärgern sich, wenn sie einen Fehler beheben und dann den nächsten gemeldet bekommen. Besser: Alle Regeln prüfen, jede Verletzung an eine Variable `$fehler` anhängen, am Ende entscheiden.

Fehler sammeln mit .= — beispiel.php

```
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>Fehler sammeln</title></head>
<body>

<?php
  $name     = trim($_POST['name'] ?? '');
  $personen = (int) ($_POST['personen'] ?? 0);

  $fehler = '';
  if ($name === '') {
    $fehler .= '<li>Bitte geben Sie Ihren Namen an.</li>';
  }
  if ($personen < 1 || $personen > 12) {
    $fehler .= '<li>Wir reservieren für 1 bis 12 Personen.</li>';
  }

  if ($fehler !== '') {
    echo "<p>Da fehlt noch etwas:</p><ul>$fehler</ul>";
  } else {
    echo "<p>Alles in Ordnung – vielen Dank!</p>";
  }
?>

</body>
</html>
```

👁 SO SIEHT ES IM BROWSER AUS

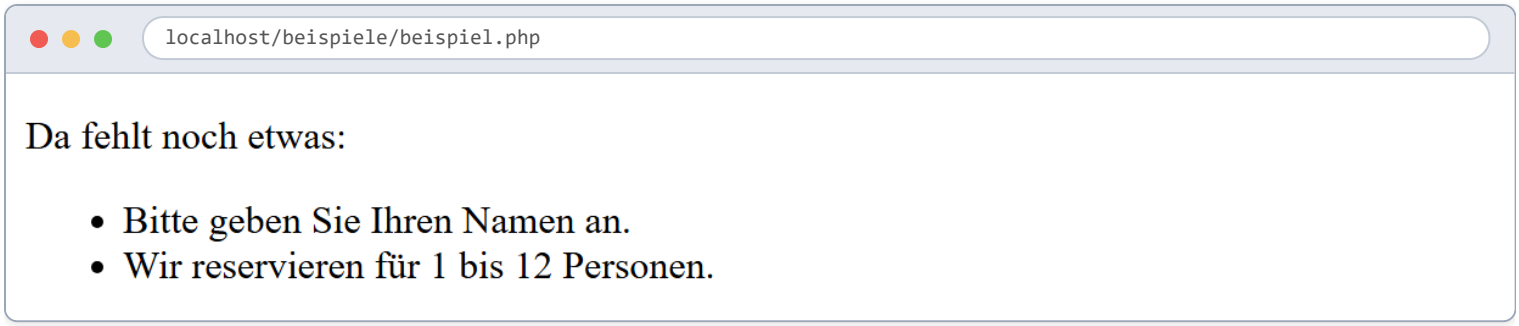


Abbildung 3: Beide Fehler werden gemeinsam gemeldet.

1. **Übernehmen** – mit `??` und `trim()`, Zahlen mit `(int)`
2. **Prüfen** – jede Regel ein eigenes `if`, Fehlermeldung mit `.=` anhängen
3. **Entscheiden** – ist `$fehler` leer, ist alles in Ordnung
